

PYCON US 2026 - 30 MIN

Breaking the Speed Limit

Fast statistical models with Python 3.14, Numba, and JAX
- told from the workflow of a statistician.

Wenxin Jiang and Jian Yin - Department of Biostatistics, City University of Hong Kong

Trust → speed → scale

Prove the statistic, accelerate the bottleneck, then test scale.

Simulation first

Controlled truth before runtime charts.

Python interface

NumPy, Numba, Python 3.14, and JAX/A100 solve different bottlenecks.

For statisticians, speed is useful only after trust

Optimization enters after reference behavior is defined, stressed, and checked.

Statistical specification evolves

Simulation exposes assumptions, hard cases, and stable outputs.

Reference code is scientific specification

Readable Python or NumPy defines the validation target.

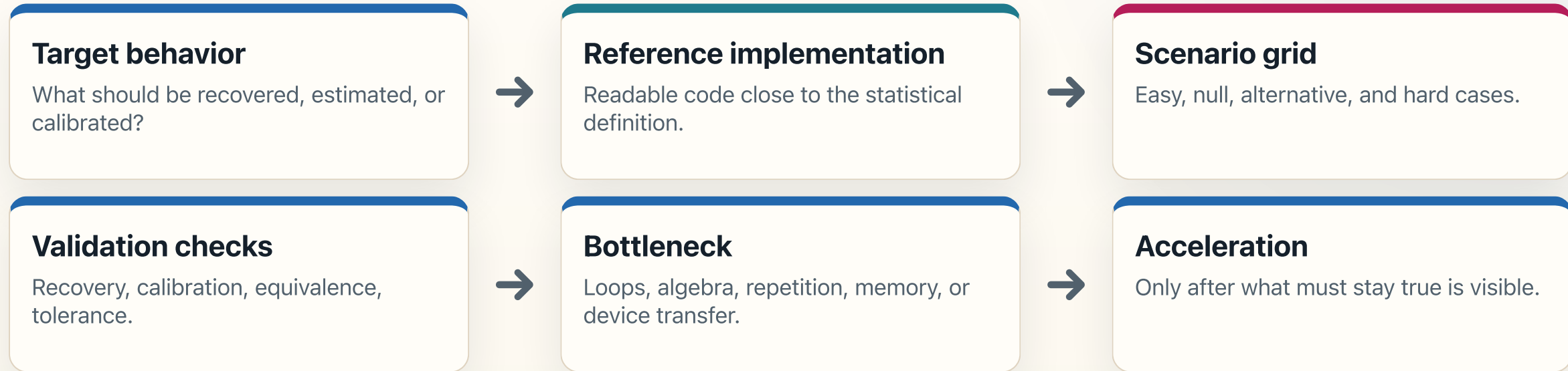
Acceleration is conditional

Faster paths must answer the same statistical question on controlled scenarios.

A speedup only counts if the optimized code answers the same statistical question.

Simulation creates an answer key

In real biomedical data, ground truth is often unavailable.



Simulation-driven statistical computing is statistical CI

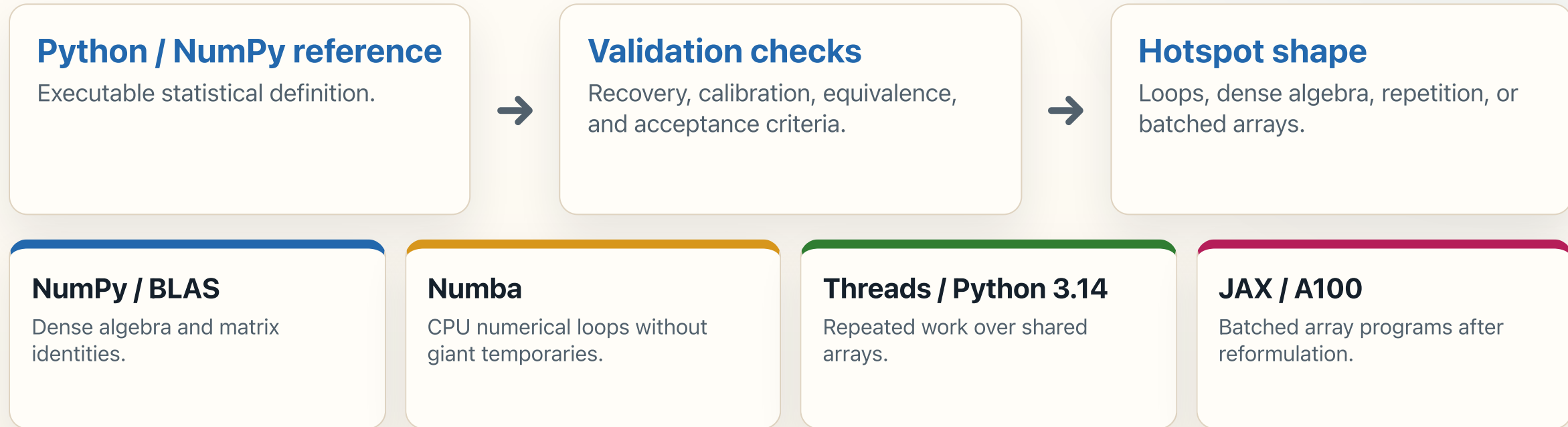
The testing instinct is familiar; the oracle is statistical behavior rather than one expected scalar.

How simulation maps to software testing

Unit tests	Fixed-seed reference equivalence.
Property tests	Null invariants and calibration.
Stress tests	Imbalance, outliers, high dimension.
Load tests	Scale N, d, K, p, R, workers, and batch sizes.
Regression tests	Acceptance criteria keep passing.

Python is the control plane; the hotspot can move to the right engine

Keep the model readable, then move the measured hotspot into the smallest engine that preserves the validation target.



Two workloads, two statistical computing shapes

k-means = iterative model fitting

assign → update centroids → repeat

Each step depends on the previous centroids.

distance loops

temporaries

compiled kernels

permutation test = resampling inference

shuffle labels → compute statistic → repeat

Independent draws become expensive across many features.

shared arrays

batching

W @ X

Evidence ladder: trust, scale, acceleration

MacBook Air

Trust tier: controlled simulation and reference equivalence.



Linux server CPU

Scale tier: larger shapes, parallelism, and memory accounting.



A100 GPU

Accelerator tier: only when the pipeline is batched and device-resident.

Correctness before runtime

Every timing row carries a validation status.

Warm timing plus memory

Cold calls, warm medians, and RSS are logged separately.

Unavailable is data

JIT, GPU sync, and interpreter state are explicitly labeled.

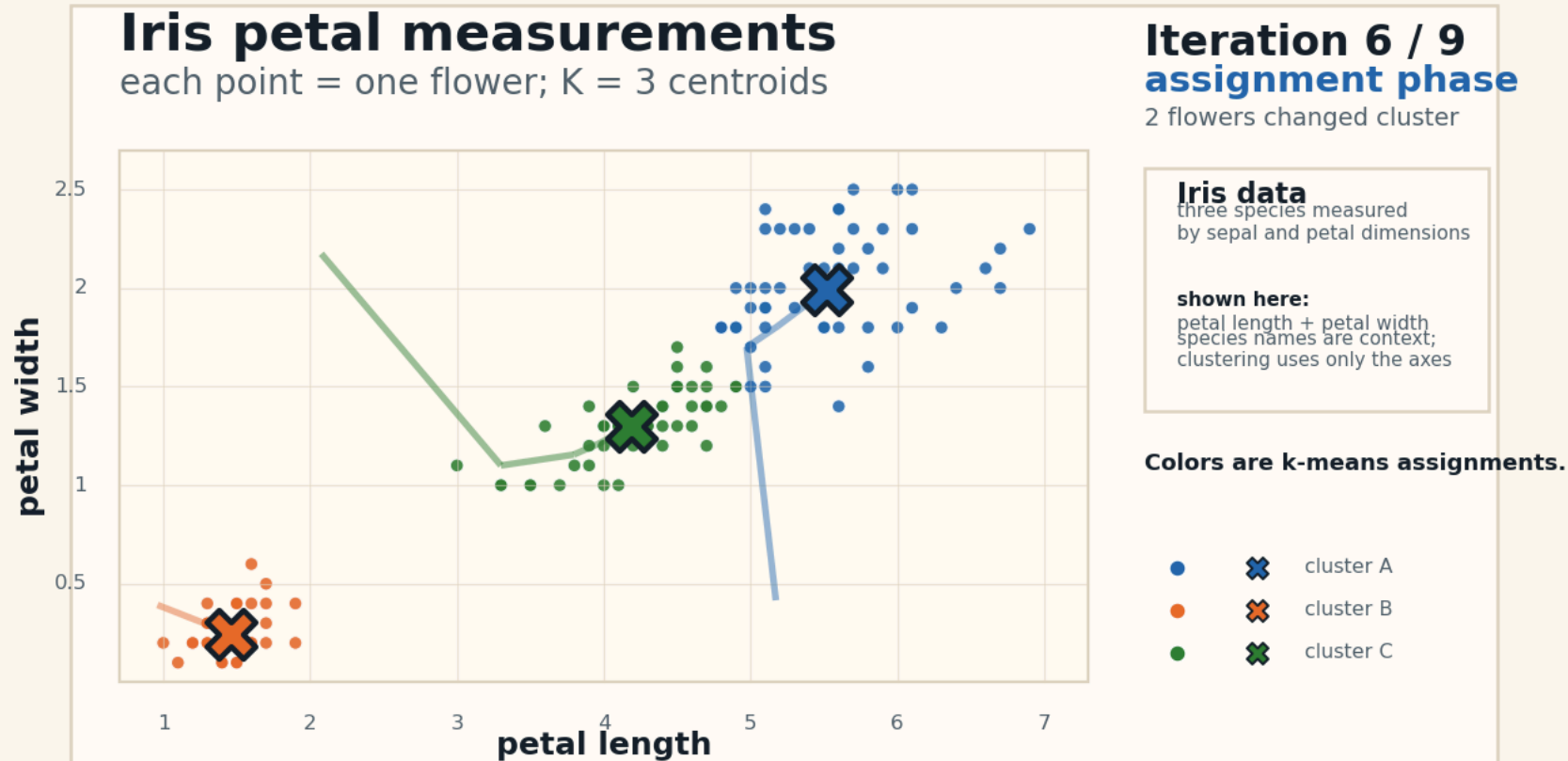
WORKLOAD 1

k-means as iterative model-fitting pressure

Same data and initialization; vary geometry and scale.

How k-means moves

Each flower is a point; k-means uses petal measurements to form 3 clusters.



One Lloyd step

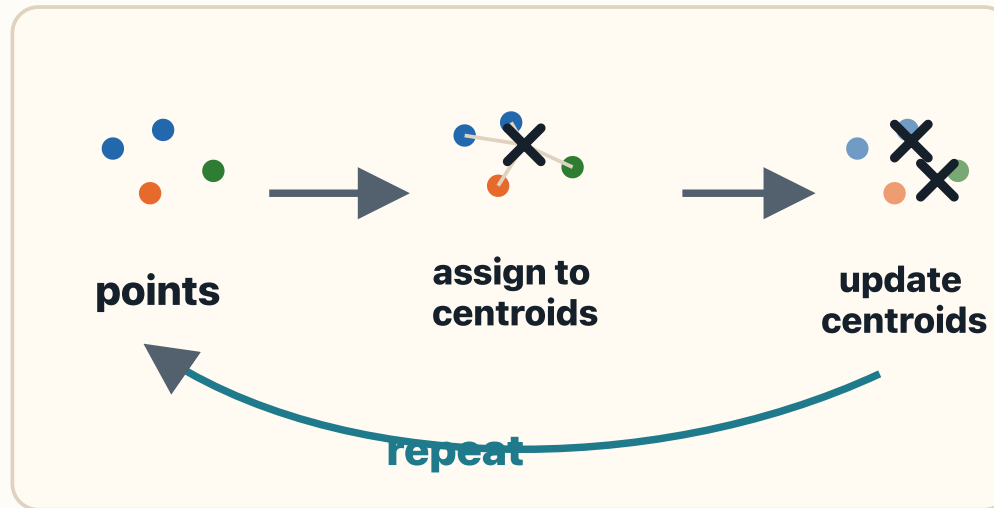
1. assign each flower to the nearest centroid
2. update centroids from assigned flowers
3. repeat until assignments stabilize

Iteration $t+1$ depends on centroids from iteration t .

Why k-means is a useful test case

A simple algorithm exposes the same pressures we see in many iterative methods.

Iterative loop



distance work per iteration
roughly scales with $N \times K \times d$

What must stay true

- same data
- same initial centroids
- same stopping rule
- comparable final inertia

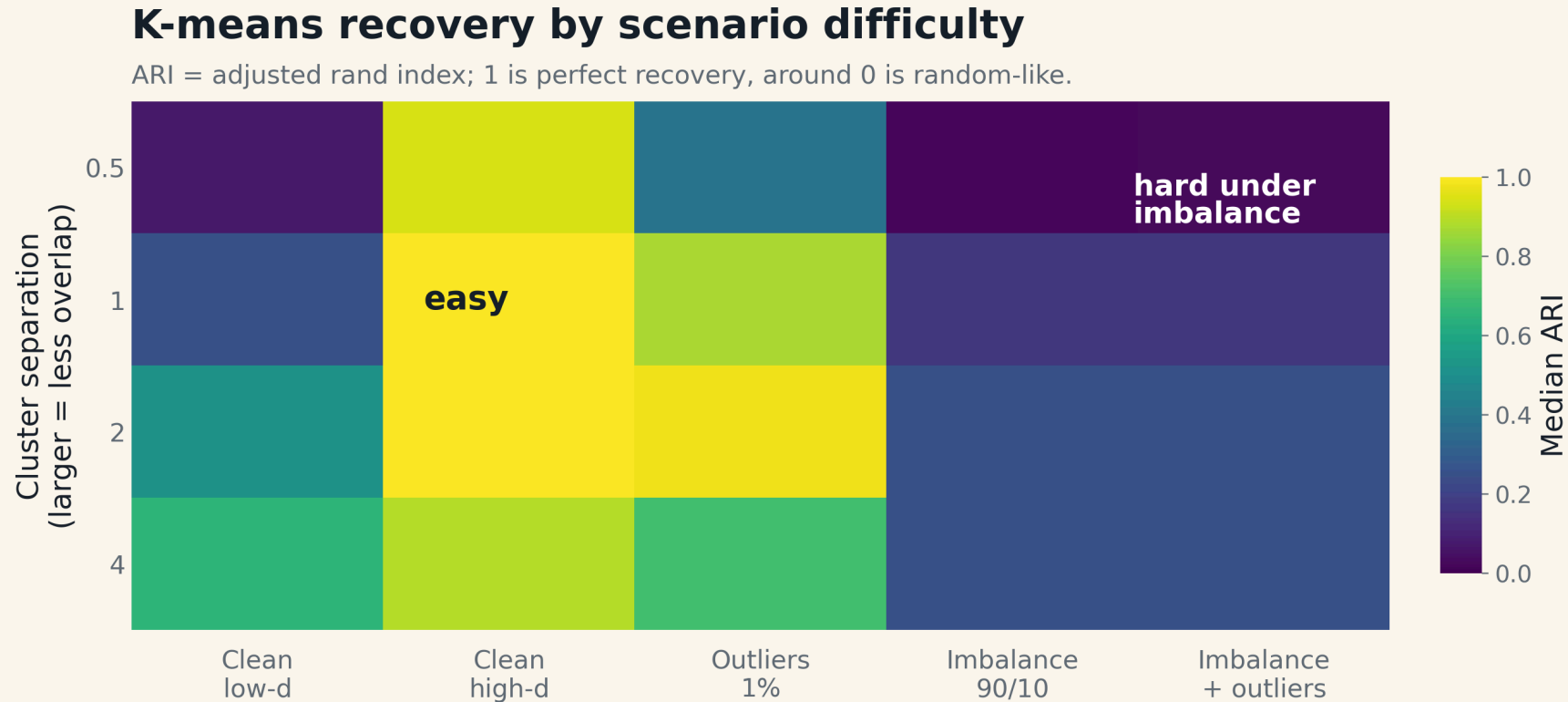
Harder data settings

- overlapping clusters
- imbalanced clusters
- outliers
- higher dimension

Why Python tools matter

- Numba: explicit loops
- NumPy/BLAS: distance algebra
- JAX/A100: large regular array work

First ask: did k-means recover the simulated clusters?



Do not use speed as a success claim where statistical recovery is poor.

Before timing: compare against the reference

Optimized k-means versions are only timed after they reproduce the same fixed-initialization result.

same data

same initial
centroids

same stopping
rule



readable reference implementation

the validation target before any speed claim



Numba / NumPy distance algebra / JAX candidates

optimized paths must preserve the same fixed-init result



compare final inertia

relative difference must stay below tolerance

MAIN CHECK

3.1e-14

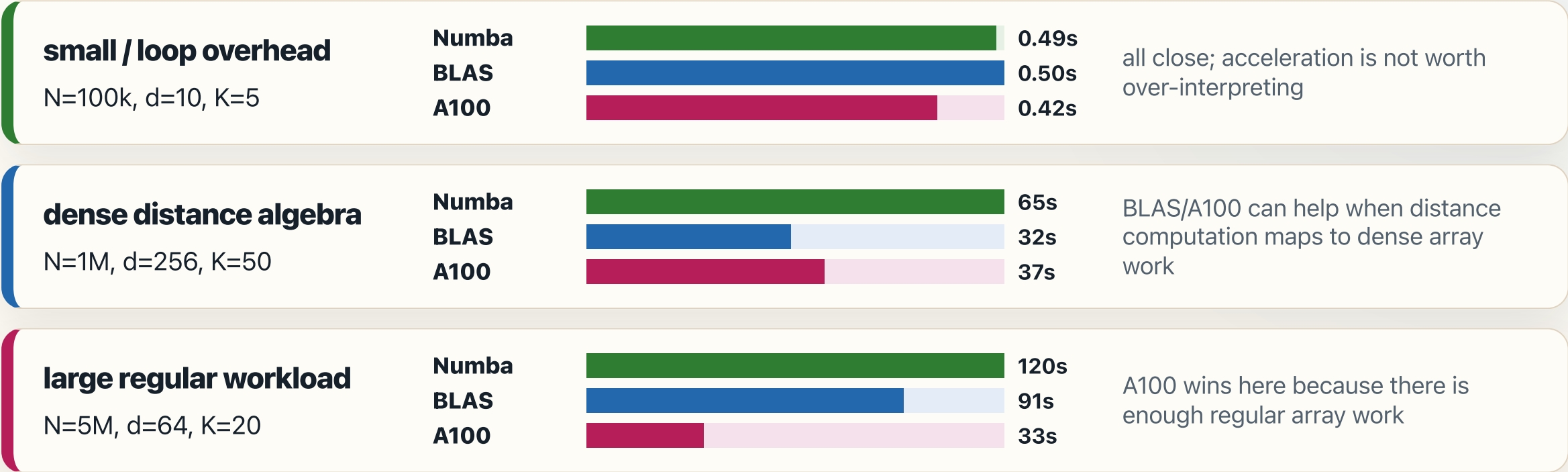
max relative inertia
difference

TOLERANCE

1e-8

Memory-risk reference rows are
skipped explicitly, not hidden.

Server k-means: the best engine depends on the shape



These are representative validated rows, not a universal ranking.

The tool choice follows the simulation signal

Unstable recovery

Do not optimize yet; the validation target is not ready.

Scalar loops

Use Numba to compile explicit CPU kernels.

Dense distance algebra

Use NumPy/BLAS when the rewrite removes large temporaries.

Huge regular work

Use JAX/A100 only after enough batched work exists.

k-means stands in for a large class of iterative statistical algorithms: EM, coordinate descent, optimization loops, and simulation-based estimators.

WORKLOAD 2

Permutation tests as resampling inference pressure

A simple inference procedure becomes expensive when we need thousands of null draws across a high-dimensional features matrix.

How a permutation test scales

One label shuffle gives one vector of feature-wise statistics; repeating shuffles builds the null distribution.

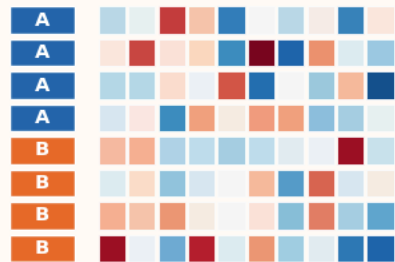
1. observed data

X matrix

rows = samples columns = features

A/B labels

features

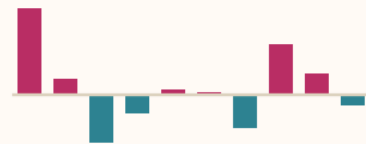


observed statistic T_{obs}

2. one permutation

A A A A B B B B

compute group
difference for
every feature



one null draw = one vector
of feature statistics

3. many permutations



null-statistics collection

streamed exceedance counts: 0 4 1 5



contrast
rows W

→ $W @ X$

matrix path appears after repeat

One resampling loop

1. shuffle group labels
2. compute feature-wise group differences
3. repeat to form the null distribution

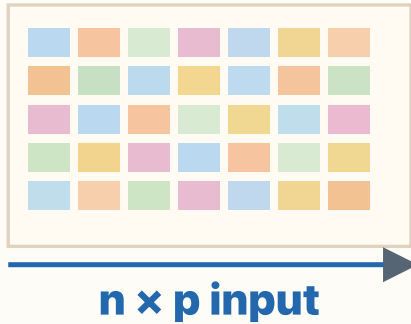
The repetitions are independent; the pipeline is not free.

Why this becomes expensive

R permutations across p features means $R \times p$ null statistics.

X matrix

n rows = samples
 p columns = features

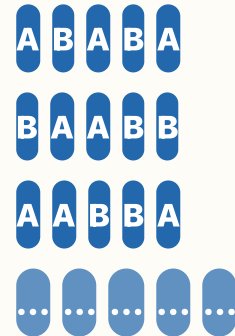


samples = patients
features = genes / biomarkers / voxels



R shuffled label vectors

R = null draws



$\times R$



$R \times p$ null statistics

or p exceedance counts after
streamed reduction



**Small statistic,
large repetition.**

Same permutations, different computation

Start with the readable loop; then rewrite the expensive repetition.

Reference loop

```
for r in range(R):  
    labels_r = same_permutation_stream[r]  
    stat[r, :] = group_difference(X, labels_r)
```

Batched / streamed path

```
for W_batch in contrast_batches(  
    same_permutation_stream):  
    T = W_batch @ X  
    counts += abs(T) >= abs(T_obs)
```

Same permutation stream. Same p-value definition. Less materialized output.

Equivalence check: loop reference vs matrix formulation

Same simulated data, same permutation stream, same p-value definition.

Context

18 workloads × 5 seeds

Reference

readable loop formulation

Candidate

matrix formulation

0.0

max |p diff|

9.4e-16

max |stat diff|

1e-6

validation tolerance

Admitted to runtime comparison.

Null calibration gate: type-I error stays near alpha

Under the null, the type-I error should stay close to nominal $\alpha = 0.05$.

Why calibration matters

Equivalence checks implementation. Calibration checks statistical behavior.

Observed check

Observed type-I error = 0.051 at nominal alpha 0.050.

What this unlocks

Now runtime comparisons become meaningful.

calibrated **0.051**
observed type-I error estimate

0.050

nominal alpha

100

null replicates

500

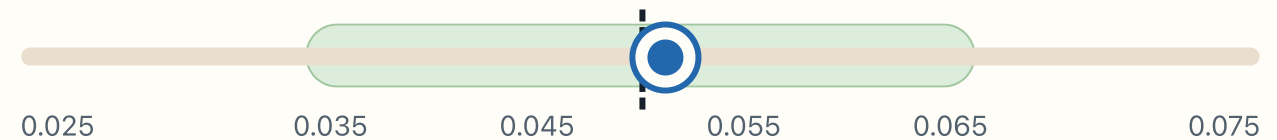
n

1,000

p and R

Calibration ruler

feature-level binomial band 0.036-0.064



Dashed: nominal 0.05

Dot: observed mean 0.051

Band is feature-level binomial variation: $0.05 \pm 1.96 * \sqrt{0.05 * 0.95 / 1000}$.

Fast code is useful only if null calibration remains correct.

LOCAL VALIDATION SCALE

Local validation scale: methods are close enough

Scope

MacBook CPU validation tier; not a server/GPU leaderboard.

Same permutations

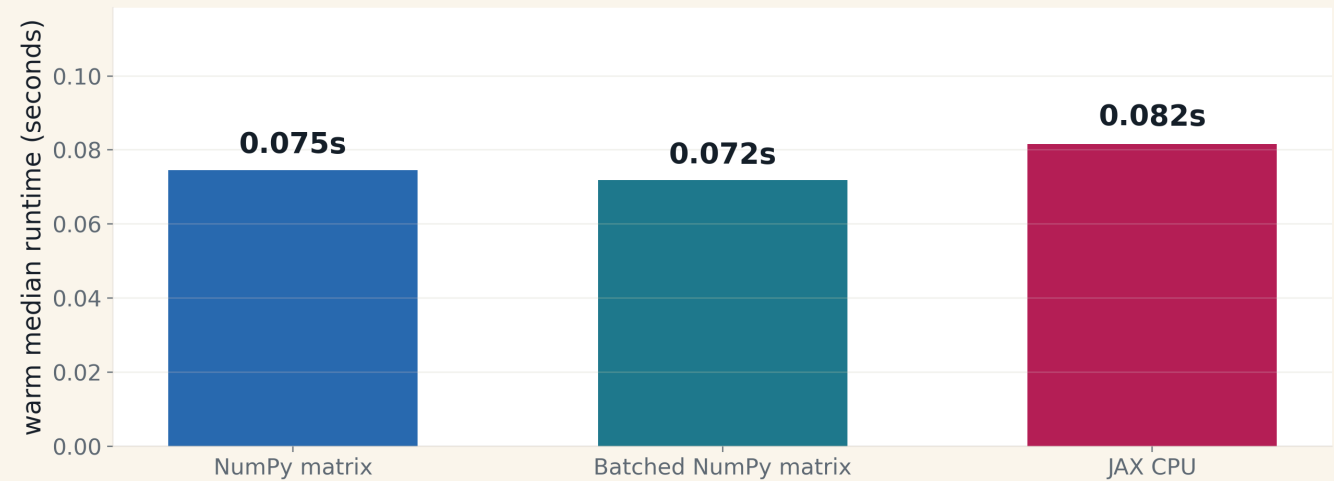
Compared only after p-value equivalence checks.

Practical rule

At validation scale, readability and correctness should dominate.

Local validation scale: methods are close enough

MacBook CPU validation tier; n=500, p=1,000, R=1,000; same validated p-values.



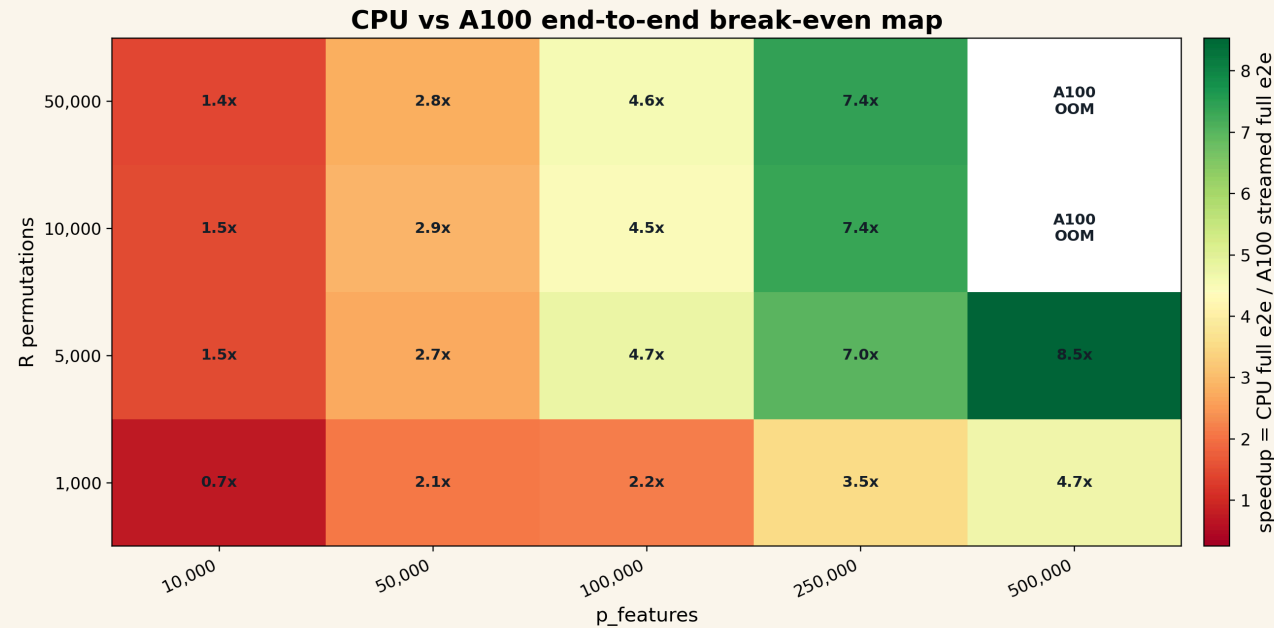
Choose the clearest correct implementation until the bottleneck is real.

At validation scale, choose the clearest correct implementation; optimize after the bottleneck is real.

DECISION MAP

When does GPU help for permutation tests?

A100 wins only after the permutation pipeline becomes large, batched, and device-resident enough.



A100 faster region appears by n=5000, p=10,000, R=5,000, batch_R=8,192.

Timing scope

Compile excluded; transfer included; kernel-only excluded.

Largest win

Largest slide-level measured speedup: 8.54x at n=5,000, p=500,000, R=5,000.

A100 can win when

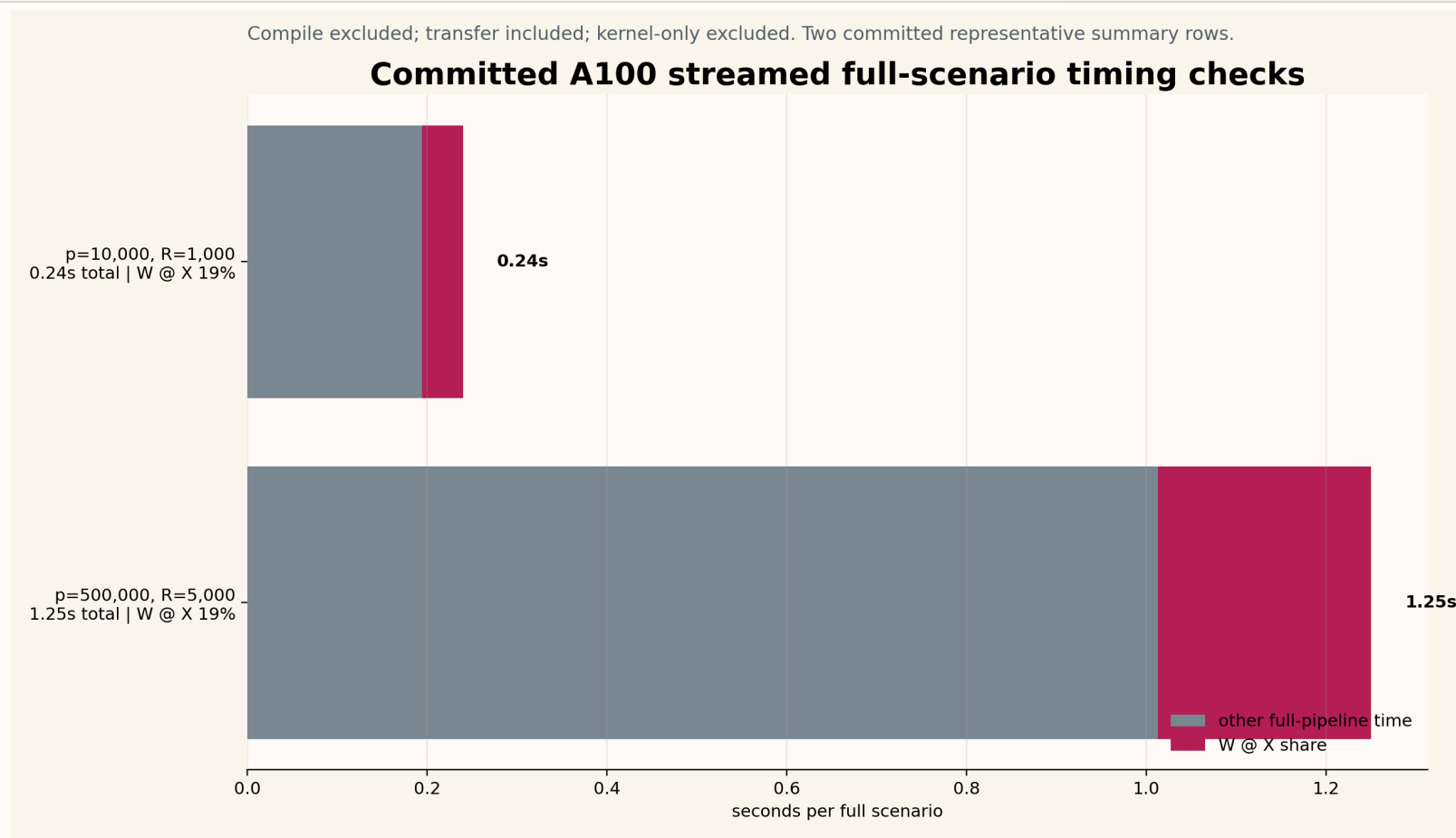
R x p is large, batch_R fits memory, streamed reduction avoids collecting R x p, and work stays on device.

speedup = matched CPU matrix baseline / A100 streamed full end-to-end.

A100 becomes faster in the streamed-reduction grid at n=5,000, p=10,000, R=5,000, batch_R=8,192.

Where does A100 time go?

Full-scenario streamed path; compile excluded, transfer included.



A fast kernel is not enough; the full pipeline decides.

PARALLELISM

CPU parallelism must be measured under resource constraints

Linux server CPU, controlled 1/4/16/64/128 sweep; high counts are shared-server evidence.

k-means

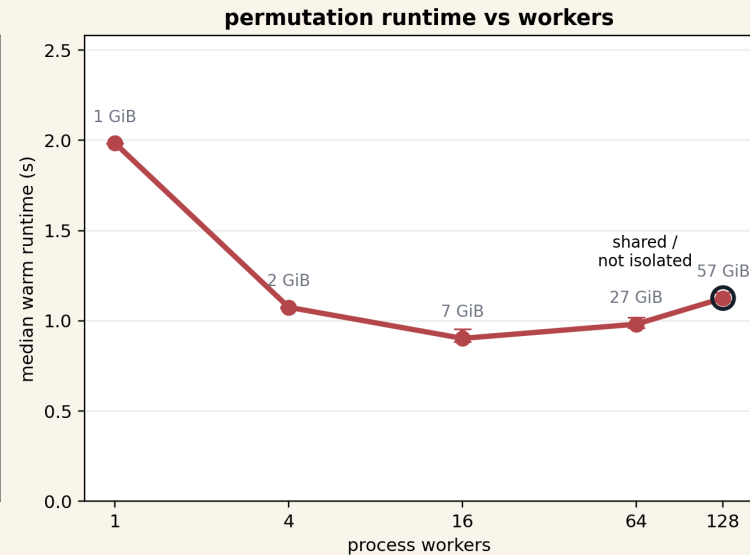
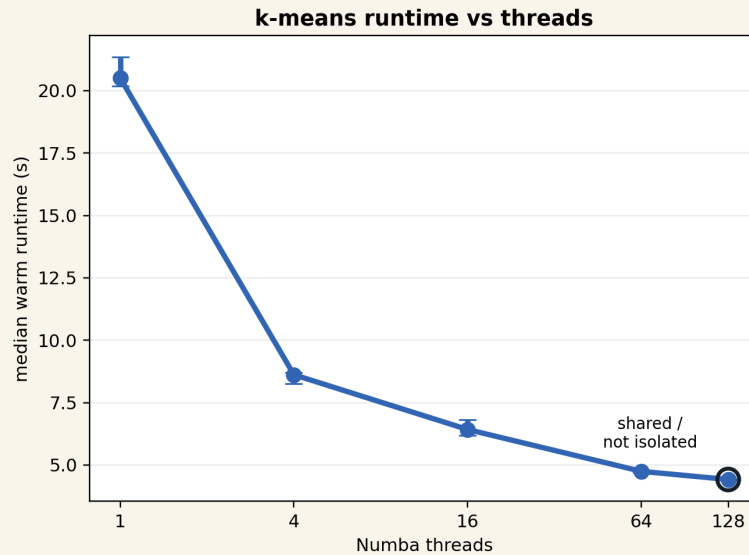
Numba kept improving through 128 in this shared-server run.

permutation

Best runtime was at 16; memory rose to about 58 GiB at 128.

128 caveat

Affinity: 512 CPUs; no exclusive allocation.



High worker counts on a shared server are hard to interpret without CPU affinity and load checks.

What each tool did in this talk

Numba

k-means
assignment/update loops

NumPy/BLAS

distance algebra

Threads/Python 3.14

repeated work over shared
arrays

JAX/A100

streamed $W @ X$ after
validation

Choose runtime after simulation identifies the bottleneck.

Codex can generate variants; the statistician owns the claims

Automation layer

- implementation variants
- scenario-grid runners
- environment metadata
- plot regeneration
- README/result manifests

Statistician-owned layer

- statistical target
- null model
- validation criteria
- interpretation of hard cases
- scientific claims

AI can automate experiment hygiene; it cannot define statistical truth.

What did the simulation reveal?

**Start from the validation
result**

**Result not
trustworthy**

keep reference / fix
method

Hot scalar loop

Numba

**Dense matrix
identity**

NumPy/BLAS

**Repeated
shared-array
work**

threads / Python 3.14

**Large batched
device-resident
work**

JAX/A100

Giant temporary

algebraic rewrite

CLOSE

Make the statistic testable. Then make the bottleneck fast.

Clear enough to trust. Fast enough to scale.

`github.com/lucajiang/FastStatisticalModels4Python`

THANK YOU

Thank you



Questions?

Clear enough to trust. Fast enough to scale.

`github.com/LucaJiang/FastStatisticalModels4Python`

Evidence map: each environment answers a different question

MacBook = validation

k-means equivalence, recovery surfaces, permutation p-value equivalence, calibration, and power.

Server CPU = scale and parallelism

k-means shape scaling, Numba thread sweep, permutation R/p scaling, worker and memory sweeps.

A100 = accelerator-shaped pipeline

JAX k-means break-even and streamed W @ X permutation follow-up.

Detailed row counts, skips, and status bookkeeping live in the result READMEs and speaker notes.

Shape stress: K and d become the bottleneck

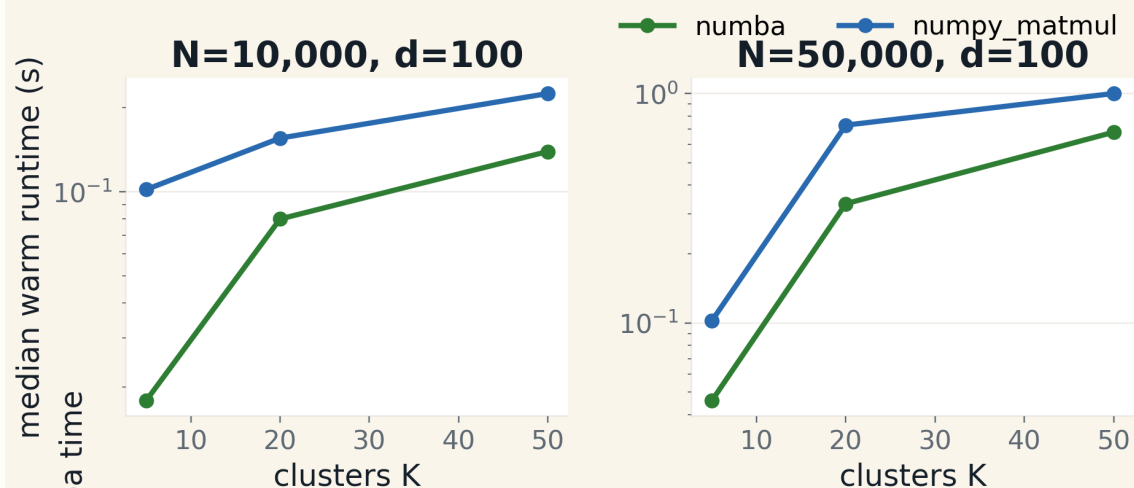
Definitions

N = number of samples

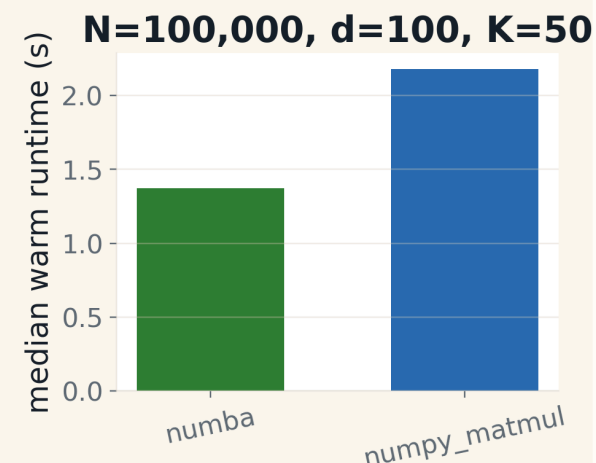
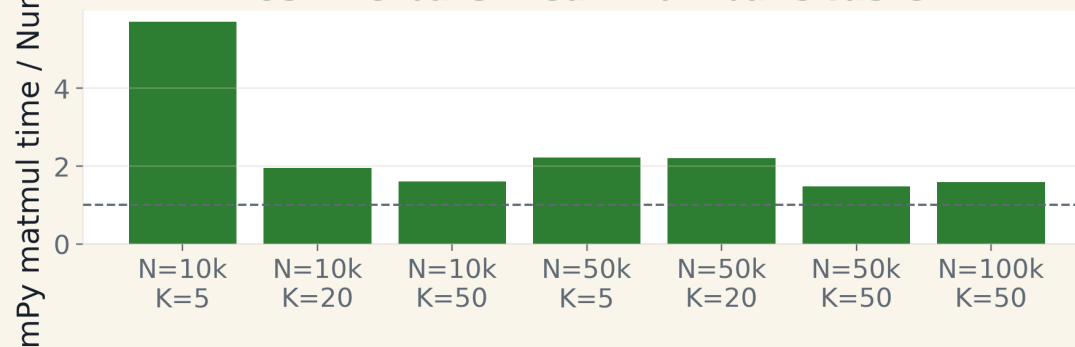
d = number of features / dimensions

K = number of clusters

k-means shape stress: speed depends on N, K, and d



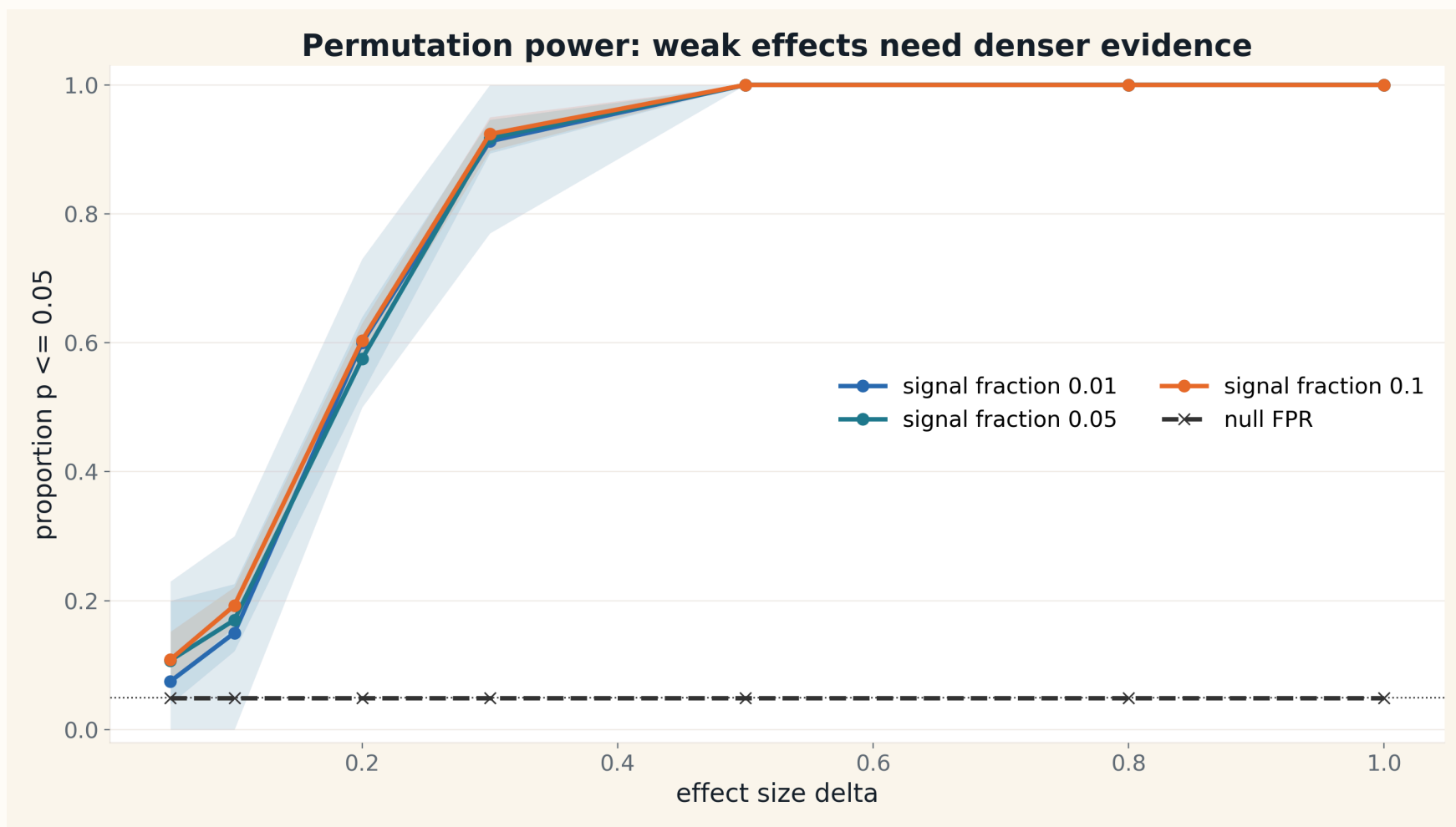
Positive bars mean Numba is faster



1,164 pass rows
max local N 100,000
d=100 stress slice
largest Numba edge 5.7x

Extra MacBook evidence: 1,164 shape-stress rows, all pass.

Power increases with effect size



Backup statistical sanity check: weak effects need denser evidence; larger effects reach high power in this local curve.

What the coding agent automated

Experiment orchestration

Built resumable runners, scenario grids, environment capture, and result manifests.

Visualization surface

Regenerated figures from CSVs and removed charts that looked precise but were unreadable.

Repository surface

Kept reproducible commands, result summaries, and QA notes together.